Laboratorio 2

Métodos supervisados: clasificación

KNN, árboles de decisión, Random Forest, XGBoost y AdaBoost

Dylan Elizondo Alvarado (504610652)
Joseph Garcia Montero (402680547)
Rodrigo Ureña Castillo (118910482)
Mariana Ureña Padilla (119180324)

EIF420O – Inteligencia Artificial, I Ciclo 2026
Universidad Nacional de Costa Rica
Profesor: Dr. Juan Murillo-Morera
15 de octubre de 2026

Resumen

Se compararon cinco familias de clasificadores (KNN, árbol de decisión, Random Forest, XGBoost y AdaBoost) en dos problemas: potabilidad del agua (2011 muestras, 9 variables fisicoquímicas) y diabetes (390 pacientes, 17 variables clínicas). Para cada familia se evaluó la configuración estándar y tres variantes que cambian un único hiperparámetro, con validación cruzada estratificada de 5 pliegues sobre el 75 % de los datos, y se probó además la selección de atributos por información mutua. El modelo elegido en cada familia se evaluó una sola vez en un 25 % de prueba reservado. En Potabilidad, Random Forest con 500 árboles obtuvo el mejor resultado (F1 macro 0.659, exactitud 70.2 %, AUC 0.727), pero ningún modelo detectó más de la mitad de las muestras potables: la señal de los datos es débil y no lineal. En Diabetes, todos los modelos basados en árboles alcanzaron F1 macro entre 0.76 y 0.78 y AUC cercanas a 0.9, dominados por la glucosa estabilizada; los siete diabéticos que ningún modelo detecta tienen glucosa en rango normal, lo que muestra un límite de la información disponible y no de los algoritmos. Todo el trabajo se integró en la clase `Clasificacion` del framework del curso y es reproducible con un solo comando por dataset.

Índice

1. Introducción y objetivos **3**

2. Fundamentos teóricos **3**

2.1. Aprendizaje supervisado y generalización 3
2.2. KNN: vecinos más cercanos 3
2.3. Árbol de decisión (DT) 4
2.4. Random Forest (RF) 4
2.5. XGBoost 4
2.6. AdaBoost 4
2.7. Evaluación: matriz de confusión y métricas 5
2.8. Validación cruzada estratificada 5

2.9. Selección de atributos por información mutua . . . 5
2.10. Desbalance de clases y ponderación . . . 5
2.11. Importancia por permutación . . . 5
2.12. Mapa conceptual . . . 6

3. Metodología **7**
3.1. Integración en el framework . . . 7
3.2. Preparación de los datos . . . 7
3.3. Protocolo experimental . . . 7

4. Resultados: Potabilidad del agua **9**
4.1. Datos y partición . . . 9
4.2. K vecinos más cercanos (KNN) . . . 10
4.3. Árbol de decisión (DT) . . . 11
4.4. Random Forest (RF) . . . 12
4.5. XGBoost . . . 13
4.6. AdaBoost . . . 13
4.7. Comparación de los cinco modelos elegidos . . . 14

5. Resultados: Diabetes **17**
5.1. Datos y partición . . . 17
5.2. K vecinos más cercanos (KNN) . . . 18
5.3. Árbol de decisión (DT) . . . 19
5.4. Random Forest (RF) . . . 20
5.5. XGBoost . . . 21
5.6. AdaBoost . . . 21
5.7. Comparación de los cinco modelos elegidos . . . 22

6. Discusión comparativa **25**
6.1. Qué variantes ganaron y por qué . . . 25
6.2. Sobreajuste y estabilidad . . . 25
6.3. Exactitud frente a F1 macro . . . 25
6.4. Limitaciones . . . 26

7. Conclusiones **26**

A. Reproducción de los experimentos **30**

1. Introducción y objetivos

El objetivo general del laboratorio es realizar experimentos de clasificación supervisada con los conjuntos de datos del curso, Potabilidad y Diabetes, usando el framework de Python construido durante el semestre. Los objetivos específicos son:

1. Explicar el funcionamiento de KNN, árboles de decisión, Random Forest, XGBoost y AdaBoost (Sección 2).
2. Definir modelos que combinen preprocesamiento, selección de atributos y cada algoritmo; para cada uno, comparar la configuración estándar con variaciones, seleccionar el mejor modelo y analizarlo (Secciones 3 a 5).

Para que las comparaciones sean justas y repetibles se fijaron tres reglas. Primero, cada variante cambia un solo hiperparámetro respecto del estándar, de modo que cualquier diferencia se pueda atribuir a ese cambio. Segundo, la elección se hace con validación cruzada sobre el conjunto de entrenamiento, y el conjunto de prueba se usa una única vez al final. Tercero, todo el preprocesamiento se aprende sólo con datos de entrenamiento, para evitar fuga de información.

El documento se organiza así: la Sección 2 presenta los fundamentos teóricos y un mapa conceptual; la Sección 3, la metodología y su implementación en el framework; las Secciones 4 y 5, los resultados y el análisis de cada dataset; la Sección 6 compara ambos casos y discute limitaciones, y la Sección 7 resume las conclusiones. El anexo contiene los comandos para reproducir todo.

2. Fundamentos teóricos

2.1. Aprendizaje supervisado y generalización

En clasificación, cada observación contiene predictoras $x_i \in \mathbb{R}^d$ y una etiqueta discreta y_i. El modelo aprende una regla $\hat{y} = f(x)$ a partir de datos de entrenamiento, y su utilidad se mide en observaciones que no participaron en ese aprendizaje [1]. Los *hiperparámetros* (por ejemplo, el número de vecinos o la profundidad de un árbol) controlan el procedimiento de ajuste; los *parámetros* se estiman con los datos. Más complejidad reduce el error de entrenamiento, pero puede aumentar la varianza ante muestras nuevas: el modelo memoriza ruido (sobreajuste). Si el modelo es demasiado simple, no captura la estructura (subajuste). Por eso hay que separar ajuste, selección y evaluación final [12, 11].

2.2. KNN: vecinos más cercanos

KNN no construye un modelo explícito: guarda las observaciones de entrenamiento y clasifica un punto nuevo según las etiquetas de sus k vecinos más próximos [2, 3]. Con la distancia de Minkowski,

$$d_p(x, z) = \Big(\sum_{j=1}^{d} |x_j - z_j|^p\Big)^{1/p}, \qquad \hat{y} = \arg\max_{c} \sum_{i \in N_k(x)} w_i \, \mathbf{1}(y_i = c),$$

donde $p = 2$ es la distancia euclidiana, $p = 1$ la Manhattan, y los pesos w_i son uniformes o inversamente proporcionales a la distancia. Un k pequeño conserva detalles locales y es sensible al ruido; uno grande suaviza la frontera y puede diluir una clase poco frecuente, porque la mayoría de los vecinos pertenecerá a la clase dominante. Como todas las variables entran en la distancia con el mismo peso, el escalado es obligatorio y los atributos irrelevantes empeoran el método: con muchas dimensiones las distancias tienden a parecerse entre sí.

2.3. Árbol de decisión (DT)

Un árbol divide recursivamente el espacio de atributos con reglas de la forma $x_j \le t$. Cada hoja es una región y predice la clase más frecuente de las observaciones de train que caen en ella. La implementación utilizada, `DecisionTreeClassifier`, sigue el algoritmo CART [4]. Para las proporciones p_c de cada clase en un nodo, las impurezas habituales son

$$G = 1 - \sum_c p_c^2 \quad \text{(Gini)}, \qquad H = -\sum_c p_c \log p_c \quad \text{(entropía)},$$

y en cada nodo se elige la división que más reduce la impureza ponderada de los hijos. Sin restricciones, el árbol crece hasta que cada hoja es pura y memoriza el entrenamiento. La profundidad máxima y el mínimo de observaciones por hoja limitan esa complejidad. Sus reglas se pueden leer directamente, pero cambios pequeños en la muestra producen árboles distintos: es un método de alta varianza.

2.4. Random Forest (RF)

Random Forest promedia muchos árboles profundos, cada uno entrenado con una muestra *bootstrap* de las filas y con un subconjunto aleatorio de atributos candidatos en cada división [5, 6]. Cada árbol individual sobreajusta, pero sus errores están poco correlacionados y el promedio reduce la varianza sin aumentar mucho el sesgo. `n_estimators` fija cuántos árboles se promedian (más árboles estabilizan la predicción pero no cambian el sesgo), y `max_depth` o `min_samples_leaf` limitan cada árbol. Es habitual que un bosque alcance exactitud casi perfecta en train sin que eso implique la misma calidad en datos nuevos.

2.5. XGBoost

XGBoost construye un ensamble *aditivo*: cada árbol nuevo corrige los errores del conjunto anterior siguiendo el gradiente de una función de pérdida, con una penalización explícita de complejidad [7, 8]:

$$F_T(x) = \sum_{t=1}^{T} \eta f_t(x), \qquad \mathcal{L} = \sum_i \ell(y_i, F_T(x_i)) + \sum_t \Omega(f_t), \qquad \Omega(f) = \gamma L + \tfrac{\lambda}{2} \sum_{j=1}^{L} w_j^2,$$

donde L es el número de hojas, w_j el valor de cada hoja y η la tasa de aprendizaje. Una η menor hace que cada árbol aporte menos y actúa como regularización; una profundidad menor limita las interacciones que cada árbol puede representar. Se usa la biblioteca `xgboost.XGBClassifier`, no el `GradientBoostingClassifier` de scikit-learn.

2.6. AdaBoost

AdaBoost combina en secuencia aprendices débiles, por defecto árboles de un solo nivel (*stumps*), y aumenta el peso de las observaciones mal clasificadas para que el siguiente aprendiz se concentre en ellas [9, 10]. En la versión binaria, con error ponderado ϵ_t,

$$\alpha_t = \tfrac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}, \qquad D_{t+1}(i) \propto D_t(i)\, e^{-\alpha_t y_i h_t(x_i)}, \qquad \hat{y} = \operatorname{sign}\Big(\sum_t \alpha_t h_t(x)\Big).$$

El número de estimadores y la tasa de aprendizaje regulan el ensamble. Si cada stump apenas supera el azar ($\epsilon_t \approx 0.5$), su peso α_t es casi nulo y el ensamble aprende muy poco; además, concentrar peso en casos difíciles puede amplificar el ruido de las etiquetas.

2.7. Evaluación: matriz de confusión y métricas

La matriz de confusión cuenta $C_{ij} = \#\{y = i, \hat{y} = j\}$ (filas reales, columnas predichas). Para cada clase c, tratada como positiva frente al resto,

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{F1 macro} = \frac{1}{|C|}\sum_c F1_c.$$

La exactitud $\sum_c C_{cc}/n$ puede ser alta aunque una clase poco frecuente no se detecte nunca. El F1 macro da el mismo peso a cada clase y por eso es la métrica de selección de este trabajo: los dos conjuntos están desbalanceados (60/40 y 85/15) [11, 18].

La **curva ROC** grafica la tasa de verdaderos positivos contra la de falsos positivos al mover el umbral de decisión sobre la probabilidad estimada. El área bajo la curva (AUC) es la probabilidad de que el modelo asigne mayor puntuación a un positivo que a un negativo elegidos al azar: 0.5 equivale al azar y 1 a una separación perfecta. No depende del umbral 0.5 que se usa para producir la matriz de confusión [19].

2.8. Validación cruzada estratificada

Una única partición de validación es muy sensible a qué filas caen en ella, sobre todo con pocos casos de la clase minoritaria. En la validación cruzada de K pliegues, el entrenamiento se divide en K partes; cada configuración se entrena K veces con $K-1$ partes y se evalúa en la restante, y se reporta la media y la desviación de la métrica [13, 14]. La versión *estratificada* conserva la proporción de clases en cada pliegue. Todo preprocesamiento (imputación, codificación, escalado y selección de atributos) se ajusta dentro de cada pliegue, para que el pliegue de evaluación nunca influya en el modelo. El conjunto de prueba se separa antes y sólo se usa una vez, con el modelo ya elegido.

2.9. Selección de atributos por información mutua

La información mutua $I(X_j; Y)$ mide cuánto se reduce la incertidumbre sobre la clase al conocer una variable; es cero si son independientes y, a diferencia de la correlación, también detecta relaciones no lineales [15, 16]. Un método de filtro como `SelectKBest` conserva las k variables con mayor $I(X_j; Y)$ [17]. Su limitación es que evalúa cada variable por separado: una variable que sólo es útil en combinación con otra puede recibir una puntuación cercana a cero.

2.10. Desbalance de clases y ponderación

Con clases desbalanceadas, un algoritmo que minimiza el error total tiende a favorecer la clase mayoritaria. Una corrección simple es ponderar cada observación de train con $w_i = \frac{n}{|C|\, n_{y_i}}$, inversamente proporcional a la frecuencia de su clase, de modo que ambas clases pesen lo mismo en el ajuste [18]. Se aplica a DT, RF, XGBoost y AdaBoost; KNN no admite pesos por observación.

2.11. Importancia por permutación

Para interpretar un modelo ya entrenado se permuta al azar una variable en el conjunto de prueba y se mide cuánto empeora la métrica. Si la caída es grande, el modelo depende de esa variable; si es nula, la ignora. Al ser independiente del algoritmo, permite comparar KNN con los métodos basados en árboles [5, 20]. No demuestra causalidad.

2.12. Mapa conceptual

La Figura 1 resume cómo se relacionan los conceptos anteriores dentro del flujo experimental.

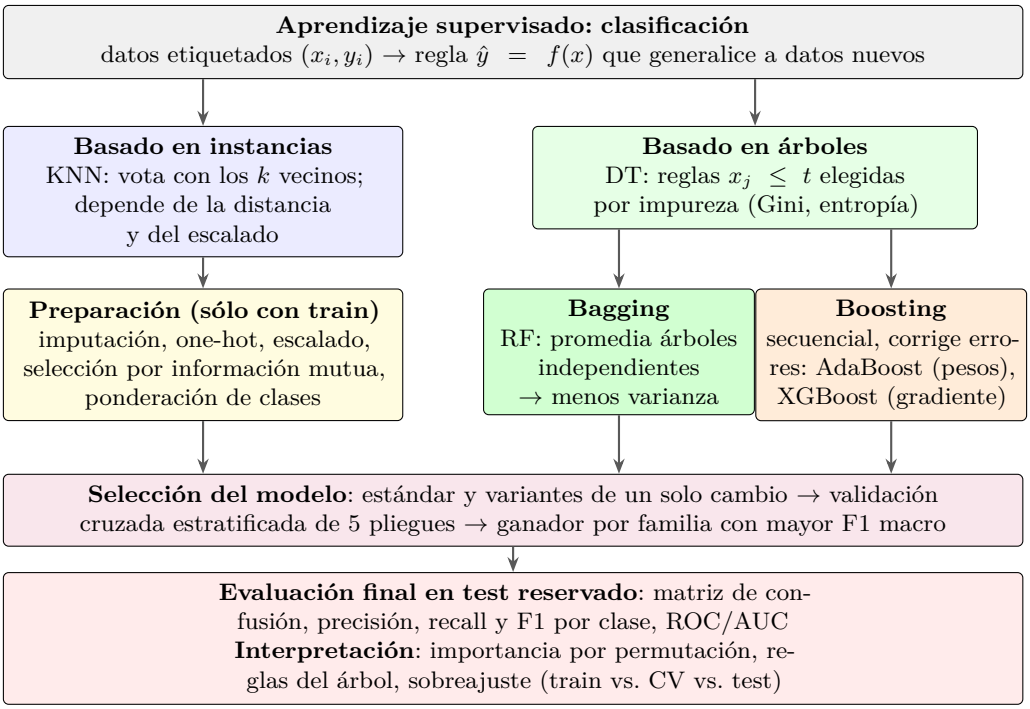


Figura 1: Mapa conceptual: familias de clasificadores, preparación sin fuga de información, selección por validación cruzada y evaluación final.

3. Metodología

3.1. Integración en el framework

Los algoritmos se implementaron en la jerarquía de clases del framework del curso: `EDA` → `Supervisado` → `Clasificacion` (módulo `framework_ia/modelos/supervisado`). `Clasificacion` expone un método por algoritmo (`knn`, `arbol_decision`, `random_forest`, `xgboost`, `adaboost`, con los alias `KNN`, `DT`, `RF`, `XGBoost`, `AdaBoost`) para entrenar una configuración, y el método `experimentar` para el protocolo completo de este informe:

```
from framework_ia.modelos import Clasificacion
modelo = Clasificacion(dataframe=datos, target="diabetes", features=predictoras)
resultado = modelo.experimentar(estrategia="cv", folds=5, random_state=42,
                                incluir_categoricas=True)
resultado.tabla            # todas las configuraciones evaluadas
resultado.mejores["RF"]    # ganador de Random Forest evaluado en test
```

El preprocesamiento vive en `PreprocesadorSupervisado`: imputación por mediana (numéricas) y moda (categóricas), codificación one-hot, estandarización y, de forma opcional, selección de los k atributos con mayor información mutua. Es un transformador de scikit-learn [21] que se ajusta en cada pliegue sólo con las filas de entrenamiento. La validación cruzada está en `validacion_cruzada.py`, y el script `scripts/ejecutar_lab02.py` ejecuta el mismo flujo desde la terminal y exporta toda la evidencia (tablas, matrices, predicciones, curvas ROC, importancias y reglas del árbol). La interfaz Streamlit del framework permite entrenar cada algoritmo de forma individual y, en la vista *Experimentos Lab02*, ejecutar este mismo protocolo con validación cruzada.

3.2. Preparación de los datos

Potabilidad. El archivo público de Water Quality [22] tiene 3276 filas, pero `ph`, `Sulfate` y `Trihalomethanes` tienen 491, 781 y 162 valores ausentes. La versión usada en clase contiene 2011 filas, que son exactamente los casos completos del archivo; por eso se trabajó con esos 2011 casos, sin imputar. El objetivo `Potability` vale 1 para agua potable y 0 para no potable.

Diabetes. El archivo original [23] tiene 403 pacientes y 19 columnas, sin una columna de clase. La clase se construyó con el criterio histórico del propio conjunto: hemoglobina glicosilada `glyhb` > 7 indica diabetes. Se excluyeron las 13 filas sin `glyhb` (no se imputa la etiqueta), y se retiraron `id` (identificador) y `glyhb` (define la clase: usarla sería fuga directa). Quedan 390 pacientes y 17 predictoras: 14 numéricas y 3 categóricas (`location`, `gender`, `frame`). Los valores ausentes de las predictoras (por ejemplo, 252 en `bp.2s` y `bp.2d`) se imputan dentro de cada ajuste.

Ambas preparaciones son deterministas, se ejecutan con un script cada una (Anexo) y registran el SHA-256 de cada archivo.

3.3. Protocolo experimental

1. **Prueba reservada.** Se separa un 25 % estratificado (semilla 42) que no interviene en ninguna decisión.

2. **Validación cruzada.** El 75 % restante se divide en 5 pliegues estratificados. Cada configuración del catálogo (Tabla 1) se entrena 5 veces y se registra la media y la desviación estándar del F1 macro en los pliegues de validación. Se elige F1 macro porque ambos datasets están desbalanceados.

3. **Variantes de un solo cambio.** Cada familia tiene su configuración estándar (valores predeterminados de la biblioteca) y tres variantes. Para DT, RF, XGBoost y AdaBoost una de ellas pondera las clases de forma inversa a su frecuencia.

4. **Selección de atributos.** La mejor configuración de cada familia se vuelve a evaluar con los mismos pliegues conservando sólo la mitad de los atributos con mayor información mutua. Se adopta la versión con mejor F1 macro medio.

5. **Modelo final.** El ganador de cada familia se reajusta con todo el entrenamiento y se evalúa una vez en prueba: matriz de confusión, métricas por clase, exactitud, F1 macro y AUC.

6. **Interpretación.** Para el ganador se calcula la importancia por permutación en prueba, las importancias internas de los árboles y, en DT, sus reglas. Estas herramientas no se usan para seleccionar.

Cuadro 1: Catálogo de configuraciones. Cada variante cambia un único aspecto respecto del estándar, para poder atribuirle su efecto.

Familia	Nombre	Parámetros solicitados	Qué cambia
KNN	`estandar`	`-`	valores predeterminados de la biblioteca
KNN	`k_15`	`n_neighbors=15`	$k = 15$ vecinos (estándar: 5)
KNN	`pesos_distancia`	`weights=distance`	votos ponderados por $1/d$ (estándar: uniformes)
KNN	`manhattan`	`p=1`	distancia Manhattan, $p = 1$ (estándar: euclidiana)
DT	`estandar`	`-`	valores predeterminados de la biblioteca
DT	`profundidad_5`	`max_depth=5`	profundidad máxima 5 (estándar: sin límite)
DT	`hoja_minima_10`	`min_samples_leaf=10`	mínimo 10 observaciones por hoja (estándar: 1)
DT	`clases_balanceadas`	`balancear_clases=True`	pesos por observación inversos a la frecuencia de su clase
RF	`estandar`	`-`	valores predeterminados de la biblioteca
RF	`500_arboles`	`n_estimators=500`	500 árboles (estándar: 100)
RF	`hoja_minima_5`	`min_samples_leaf=5`	mínimo 5 observaciones por hoja (estándar: 1)
RF	`clases_balanceadas`	`balancear_clases=True`	pesos por observación inversos a la frecuencia de su clase
XGBoost	`estandar`	`-`	valores predeterminados de la biblioteca
XGBoost	`tasa_0_05`	`learning_rate=0.05`	tasa de aprendizaje 0.05 (estándar: 0.3)
XGBoost	`profundidad_3`	`max_depth=3`	profundidad máxima 3 (estándar: 6)
XGBoost	`clases_balanceadas`	`balancear_clases=True`	pesos por observación inversos a la frecuencia de su clase
AdaBoost	`estandar`	`-`	valores predeterminados de la biblioteca
AdaBoost	`200_estimadores`	`n_estimators=200`	200 estimadores (estándar: 50)
AdaBoost	`tasa_0_5`	`learning_rate=0.5`	tasa de aprendizaje 0.5 (estándar: 1.0)
AdaBoost	`clases_balanceadas`	`balancear_clases=True`	pesos por observación inversos a la frecuencia de su clase

Cuadro 2: Tamaños efectivos del protocolo (semilla 42).

Dataset	Filas	Train	Test	Validación por pliegue	Predictoras	Atributos transf.	k sel.
Potabilidad del agua	2011	1508	503	302/302/302/301/301	9	9	5
Diabetes	390	292	98	59/59/58/58/58	17	21	11

4. Resultados: Potabilidad del agua

4.1. Datos y partición

Se analizan 2011 muestras de agua con nueve mediciones fisicoquímicas (`ph`, dureza, sólidos disueltos, cloraminas, sulfato, conductividad, carbono orgánico, trihalometanos y turbidez). El 59.7 % de las muestras es no potable. Un rasgo central de este conjunto es que las medias de cada variable son casi idénticas en las dos clases (por ejemplo, `ph` 7.07 frente a 7.11 y sulfato 333.7 frente a 332.5 mg/L): ninguna variable separa las clases por sí sola. Lo que cambia es la dispersión y los extremos. El 73 % de las muestras con sulfato menor que 260 mg/L y el 62 % de las que superan 420 mg/L son potables, frente al 35 % entre 300 y 340 mg/L. Con el `ph` ocurre algo parecido: la proporción de potables es mayor cerca de la neutralidad (43–45 % entre 6 y 8) que en los extremos (30 % con `ph` ≤ 5). Son relaciones en forma de U, no lineales, y eso condiciona qué algoritmos funcionan mejor.

Cuadro 3: Procedencia de Potabilidad del agua.

Elemento	Valor
Archivo analizado	`potabilidad.csv`
Original	`water_potability.csv`, 3276 filas
Filas analizadas	2011
Objetivo	`Potability`
Predictoras	9
Fuente	`https://www.kaggle.com/datasets/adityakadiwal/water-potability`
SHA-256 analizado	`d2569bece50025b9dbf21cfd720ecbed cdaa838ca082db29ef853c5fb9da8e12`

Cuadro 4: Distribución de clases en Potabilidad del agua (partición estratificada 75/25, semilla 42).

Clase	Train	Test	Total
No potable (0)	900	300	1200
Potable (1)	608	203	811
Total	1508	503	2011

Una referencia trivial que siempre predice la clase mayoritaria de train (No potable (0)) obtendría en test una exactitud de 59.6 % y un F1 macro de 0.374. Cualquier modelo útil debe superar con claridad ese F1 macro, no sólo la exactitud.

4.2. K vecinos más cercanos (KNN)

Cuadro 5: K vecinos más cercanos (KNN) en Potabilidad del agua: validación cruzada estratificada de 5 pliegues sobre train. ⋆ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.5892 ± 0.029	–	62.6 %	0.5901
`k_15`	$k = 15$ vecinos (estándar: 5)	0.6000 ± 0.030	+0.011	65.6 %	0.6066
`pesos_distancia`	votos ponderados por $1/d$ (estándar: uniformes)	0.5877 ± 0.030	-0.001	62.5 %	0.5887
`manhattan` ⋆	distancia Manhattan, $p = 1$ (estándar: euclidiana)	0.6036 ± 0.030	+0.014	64.1 %	0.6042
`manhattan+seleccion_k5`	mejor anterior + 5 atributos por información mutua	0.5364 ± 0.033	-0.053	57.7 %	0.5399

Modelo final. Configuración `manhattan` con todos los atributos, reajustada con las 1508 filas de train. F1 macro: 0.7472 en entrenamiento, 0.6036 en validación cruzada y 0.5969 en test (exactitud 63.4 %, AUC 0.668).

Cuadro 6: K vecinos más cercanos (KNN): matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	No potable (0)	Potable (1)
No potable (0)	236	64
Potable (1)	120	83

Cuadro 7: K vecinos más cercanos (KNN): métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
No potable (0)	300	0.6629	0.7867	0.7195
Potable (1)	203	0.5646	0.4089	0.4743

Análisis. Las cuatro configuraciones quedan dentro de una desviación estándar entre sí (0.588–0.604, con desviaciones cercanas a 0.03), así que la elección de la distancia Manhattan (+0.014 sobre el estándar) es una mejora pequeña y poco segura. Aumentar a $k = 15$ sube la exactitud de validación de 62.6 % a 65.7 % pero apenas el F1 macro: promediar más vecinos suaviza la frontera hacia la clase mayoritaria. Ponderar por distancia no cambia nada relevante. La selección de cinco atributos empeoró el F1 macro a 0.536, por la razón que se discute en la síntesis. En prueba, KNN reconoce bien el agua no potable (recall 0.787) pero sólo 83 de 203 muestras potables (recall 0.409). La importancia por permutación reparte el efecto entre `ph`, sulfato y sólidos, sin una variable dominante. Esto es coherente con el método: KNN usa las nueve variables con el mismo peso, aunque la mayoría aporta poca información, y esas dimensiones irrelevantes vuelven las distancias menos informativas. La diferencia entre F1 de entrenamiento (0.747) y de prueba (0.597) muestra además que, con $k = 5$, el modelo depende mucho de los vecinos concretos.

4.3. Árbol de decisión (DT)

Cuadro 8: Árbol de decisión (DT) en Potabilidad del agua: validación cruzada estratificada de 5 pliegues sobre train. ⋆ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.5654 ± 0.048	–	57.5 %	0.5681
`profundidad_5`	profundidad máxima 5 (estándar: sin límite)	0.5346 ± 0.071	-0.031	62.5 %	0.5667
`hoja_minima_10` ⋆	mínimo 10 observaciones por hoja (estándar: 1)	0.5908 ± 0.047	+0.025	61.0 %	0.5906
`clases_balanceadas`	pesos por observación inversos a la frecuencia de su clase	0.5633 ± 0.049	-0.002	58.4 %	0.5633
`hoja_minima_10+ seleccion_k5`	mejor anterior + 5 atributos por información mutua	0.5487 ± 0.050	-0.017	57.6 %	0.5499

Modelo final. Configuración `hoja_minima_10` con todos los atributos, reajustada con las 1508 filas de train. F1 macro: 0.7935 en entrenamiento, 0.5908 en validación cruzada y 0.5777 en test (exactitud 61.2 %, AUC 0.657).

Cuadro 9: Árbol de decisión (DT): matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	No potable (0)	Potable (1)
No potable (0)	226	74
Potable (1)	121	82

Cuadro 10: Árbol de decisión (DT): métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
No potable (0)	300	0.6513	0.7533	0.6986
Potable (1)	203	0.5256	0.4039	0.4568

```
|--- Sulfate <= 261.02
|   |--- Solids <= 18346.62
|   |   |--- clase: 0 [n=13]
|   |--- Solids > 18346.62
|   |   |--- ph <= 6.23
|   |   |   |--- clase: 0 [n=10]
|   |   |--- ph > 6.23
|   |   |   |--- clase: 1 [n=39] (rama truncada)
|--- Sulfate > 261.02
|   |--- Sulfate <= 387.27
|   |   |--- ph <= 6.27
|   |   |   |--- clase: 0 [n=374] (rama truncada)
|   |   |--- ph > 6.27
|   |   |   |--- clase: 0 [n=939] (rama truncada)
|   |--- Sulfate > 387.27
|   |   |--- ph <= 7.91
|   |   |   |--- clase: 1 [n=93] (rama truncada)
|   |   |--- ph > 7.91
|   |   |   |--- clase: 0 [n=40] (rama truncada)
```

Figura 2: Primeros tres niveles del árbol final (profundidad total 14, 107 hojas). Los umbrales están en las unidades originales: la estandarización se invirtió con la media y desviación de train. n es la cantidad de observaciones de train en el nodo.

Análisis. El árbol sin restricciones (estándar) obtiene F1 macro 0.565 con una exactitud de apenas 57.5 %, menor que la de predecir siempre "no potable": crece hasta memorizar el entrenamiento. Limitar la profundidad a 5 sube la exactitud (62.5 %) pero baja el F1 macro (0.535), porque un árbol tan corto termina prediciendo casi siempre la clase mayoritaria. La mejor opción es exigir al menos 10 observaciones por hoja (0.591, +0.025): regulariza sin impedir que el árbol aprenda las

reglas en U. El árbol final sigue siendo grande (profundidad 14, 107 hojas) y el F1 cae de 0.794 en entrenamiento a 0.578 en prueba, lo que confirma su alta varianza. Sus primeras reglas son interpretables y coinciden con lo observado en los datos: con sulfato ≤ 261 mg/L, sólidos mayores que 18 347 mg/L y `ph` > 6.23 predice potable; con sulfato entre 261 y 387 mg/L predice no potable, y por encima de 387 mg/L vuelve a predecir potable cuando `ph` ≤ 7.91. El árbol representa la forma en U combinando dos cortes sobre la misma variable y una interacción con el `ph`.

4.4. Random Forest (RF)

Cuadro 11: Random Forest (RF) en Potabilidad del agua: validación cruzada estratificada de 5 pliegues sobre train. $\star$ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.6287 ± 0.031	–	67.7 %	0.6310
`500_arboles` $\star$	500 árboles (estándar: 100)	0.6435 ± 0.022	+0.015	69.0 %	0.6442
`hoja_minima_5`	mínimo 5 observaciones por hoja (estándar: 1)	0.6151 ± 0.022	-0.014	67.2 %	0.6209
`clases_balanceadas`	pesos por observación inversos a la frecuencia de su clase	0.6278 ± 0.045	-0.001	65.5 %	0.6262
`500_arboles+ seleccion_k5`	mejor anterior + 5 atributos por información mutua	0.5436 ± 0.039	-0.085	60.1 %	0.5538

Modelo final. Configuración `500_arboles` con todos los atributos, reajustada con las 1508 filas de train. F1 macro: 1.0000 en entrenamiento, 0.6435 en validación cruzada y 0.6585 en test (exactitud 70.2 %, AUC 0.727).

Cuadro 12: Random Forest (RF): matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	No potable (0)	Potable (1)
No potable (0)	266	34
Potable (1)	116	87

Cuadro 13: Random Forest (RF): métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
No potable (0)	300	0.6963	0.8867	0.7801
Potable (1)	203	0.7190	0.4286	0.5370

Análisis. Random Forest obtiene el mejor resultado de este dataset. El estándar (100 árboles) alcanza 0.629 y pasar a 500 árboles sube el F1 macro a 0.644 y además baja la desviación entre pliegues de 0.031 a 0.022: con más árboles el promedio es más estable, que es exactamente lo que predice la teoría del bagging. Exigir 5 observaciones por hoja empeora (0.615), porque limita los árboles profundos que el bosque necesita para capturar las interacciones. Ponderar las clases no ayudó. En prueba logra exactitud de 70.2 % (contra 59.6 % de la referencia), F1 macro 0.659 y AUC 0.727. Es el modelo más preciso al predecir potable (0.719: de cada 10 muestras que marca como potables, 7 lo son), pero sigue dejando pasar 116 de las 203 muestras potables (recall 0.429). El F1 de entrenamiento es 1.000: cada árbol memoriza su muestra, y aun así el promedio generaliza mejor que cualquier modelo individual. La importancia por permutación (Figura 6) identifica el sulfato (caída de 0.131 en F1) y el `ph` (0.096) como las variables decisivas, seguidas de cloraminas, dureza y sólidos.

4.5. XGBoost

Cuadro 14: XGBoost en Potabilidad del agua: validación cruzada estratificada de 5 pliegues sobre train. ⋆ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.6220 ± 0.025	–	65.1 %	0.6210
`tasa_0_05`	tasa de aprendizaje 0.05 (estándar: 0.3)	0.6172 ± 0.024	-0.005	66.3 %	0.6196
`profundidad_3`	profundidad máxima 3 (estándar: 6)	0.6208 ± 0.034	-0.001	65.1 %	0.6197
`clases_balanceadas` ⋆	pesos por observación inversos a la frecuencia de su clase	0.6392 ± 0.044	+0.017	65.8 %	0.6375
`clases_balanceadas+ seleccion_k5`	mejor anterior + 5 atributos por información mutua	0.5439 ± 0.012	-0.078	56.6 %	0.5441

Modelo final. Configuración `clases_balanceadas` con todos los atributos, reajustada con las 1508 filas de train. F1 macro: 1.0000 en entrenamiento, 0.6392 en validación cruzada y 0.5896 en test (exactitud 61.0 %, AUC 0.671).

Cuadro 15: XGBoost: matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	No potable (0)	Potable (1)
No potable (0)	210	90
Potable (1)	106	97

Cuadro 16: XGBoost: métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
No potable (0)	300	0.6646	0.7000	0.6818
Potable (1)	203	0.5187	0.4778	0.4974

Análisis. Las variantes de tasa de aprendizaje (0.05) y profundidad (3) quedan prácticamente iguales al estándar (0.617–0.622). La única mejora es ponderar las clases (0.639, +0.017), que obliga al modelo a prestar más atención a las muestras potables. En prueba esto se ve como el segundo mayor recall de potables (0.478, 97 de 203), a costa de más falsos positivos (90 frente a 34 de RF) y una exactitud de 61.0 %. XGBoost es el modelo que más cae de validación (0.639) a prueba (0.590), y su F1 de entrenamiento es 1.000: con la tasa 0.3 y profundidad 6 predeterminadas, los 100 árboles alcanzan a memorizar el entrenamiento. En este dataset ruidoso, el boosting por gradiente termina ajustando ruido, mientras que el promedio de RF lo atenúa.

4.6. AdaBoost

Cuadro 17: AdaBoost en Potabilidad del agua: validación cruzada estratificada de 5 pliegues sobre train. ⋆ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.4714 ± 0.018	–	59.7 %	0.5224
`200_estimadores`	200 estimadores (estándar: 50)	0.5014 ± 0.017	+0.030	59.9 %	0.5337
`tasa_0_5`	tasa de aprendizaje 0.5 (estándar: 1.0)	0.4501 ± 0.016	-0.021	61.5 %	0.5286
`clases_balanceadas` ⋆	pesos por observación inversos a la frecuencia de su clase	0.5404 ± 0.039	+0.069	55.2 %	0.5456
`clases_balanceadas+ seleccion_k5`	mejor anterior + 5 atributos por información mutua	0.5281 ± 0.011	+0.057	54.4 %	0.5314

Modelo final. Configuración `clases_balanceadas` con todos los atributos, reajustada con las 1508 filas de train. F1 macro: 0.5747 en entrenamiento, 0.5404 en validación cruzada y 0.5822 en test (exactitud 59.8 %, AUC 0.615).

Cuadro 18: AdaBoost: matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	No potable (0)	Potable (1)
No potable (0)	200	100
Potable (1)	102	101

Cuadro 19: AdaBoost: métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
No potable (0)	300	0.6623	0.6667	0.6645
Potable (1)	203	0.5025	0.4975	0.5000

Análisis. AdaBoost estándar tiene el peor desempeño del estudio: F1 macro 0.471 con exactitud 59.7 %, igual a la proporción de la clase mayoritaria. Es decir, casi siempre predice "no potable". Bajar la tasa a 0.5 lo empeora (0.450) y duplicar los estimadores mejora poco (0.501). La razón está en sus aprendices: cada stump es un único corte sobre una variable, y como las medias de ambas clases son casi iguales, ningún corte supera por mucho el azar. En ese caso el peso α_t de cada stump es pequeño y el ensamble no logra representar la interacción entre sulfato y `ph`. La ponderación de clases es la variante con mayor efecto de todo el dataset (+0.069, F1 0.540): al igualar el peso de ambas clases, los stumps dejan de favorecer la mayoritaria. En prueba alcanza el recall más alto de potables (0.498), pero con precisión 0.502 y AUC 0.615, la más baja. A diferencia de RF y XGBoost no sobreajusta (F1 0.575 en entrenamiento y 0.582 en prueba): su problema es de sesgo (subajuste), no de varianza. Casi toda su capacidad se apoya en el sulfato.

4.7. Comparación de los cinco modelos elegidos

Cuadro 20: Potabilidad del agua: resumen de los ganadores. La selección usó sólo la columna F1 CV; el mejor algoritmo por validación es RF.

Familia	Configuración	F1 train	F1 CV	F1 test	Exactitud test	Recall Potable (1)	AUC
KNN	`manhattan`	0.7472	0.6036	0.5969	63.4 %	0.4089	0.668
DT	`hoja_minima_10`	0.7935	0.5908	0.5777	61.2 %	0.4039	0.657
RF	`500_arboles`	1.0000	0.6435	0.6585	70.2 %	0.4286	0.727
XGBoost	`clases_balanceadas`	1.0000	0.6392	0.5896	61.0 %	0.4778	0.671
AdaBoost	`clases_balanceadas`	0.5747	0.5404	0.5822	59.8 %	0.4975	0.615

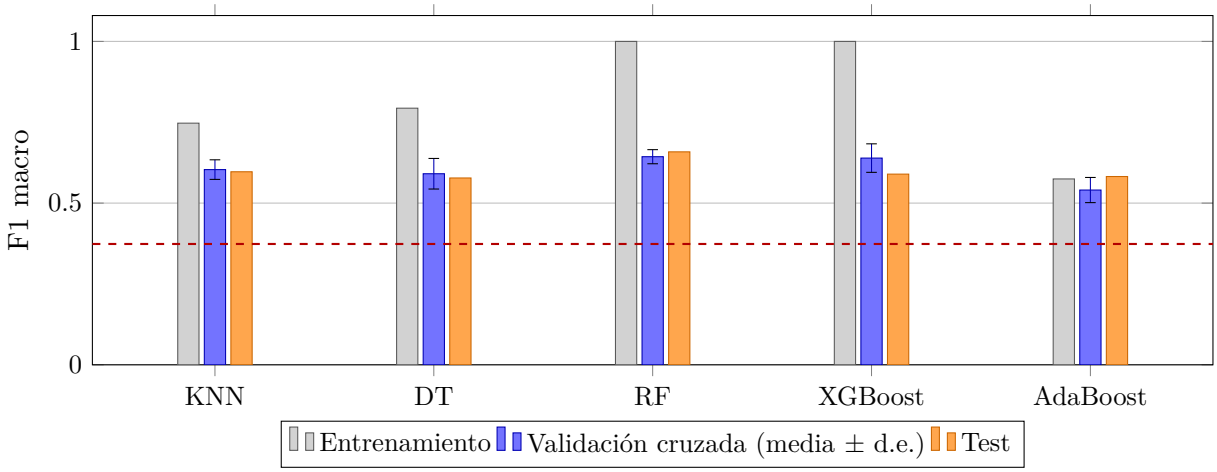


Figura 3: Potabilidad del agua: F1 macro del modelo elegido en cada familia. La línea roja es la referencia que siempre predice la clase mayoritaria (F1 macro 0.374). La distancia entre la barra gris y la azul indica sobreajuste.

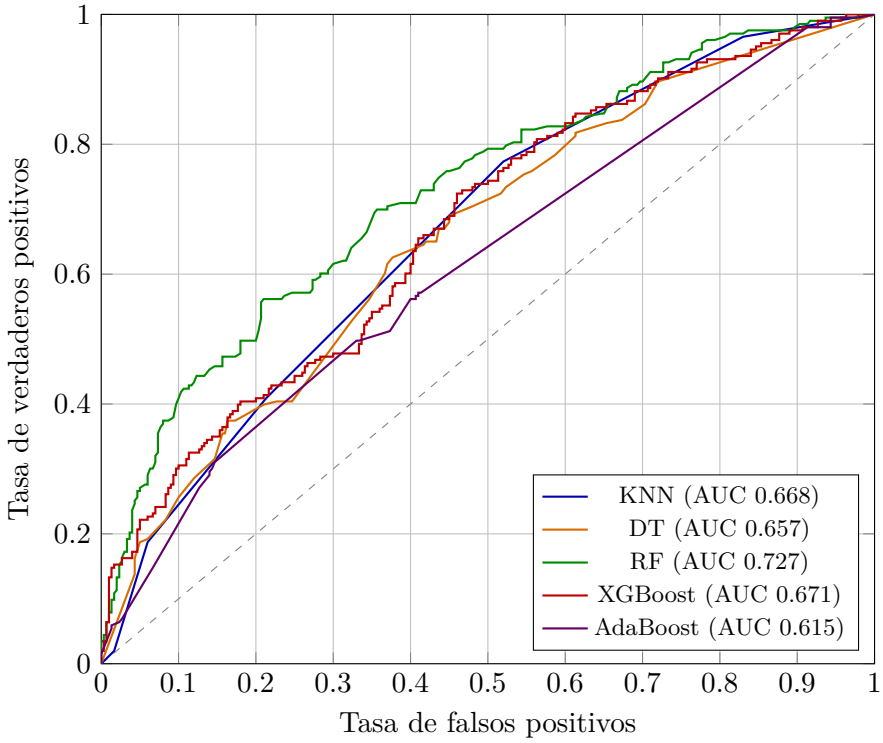


Figura 4: Potabilidad del agua: curvas ROC en test de los cinco modelos elegidos; clase positiva: Potable (1). La diagonal corresponde a un clasificador aleatorio.

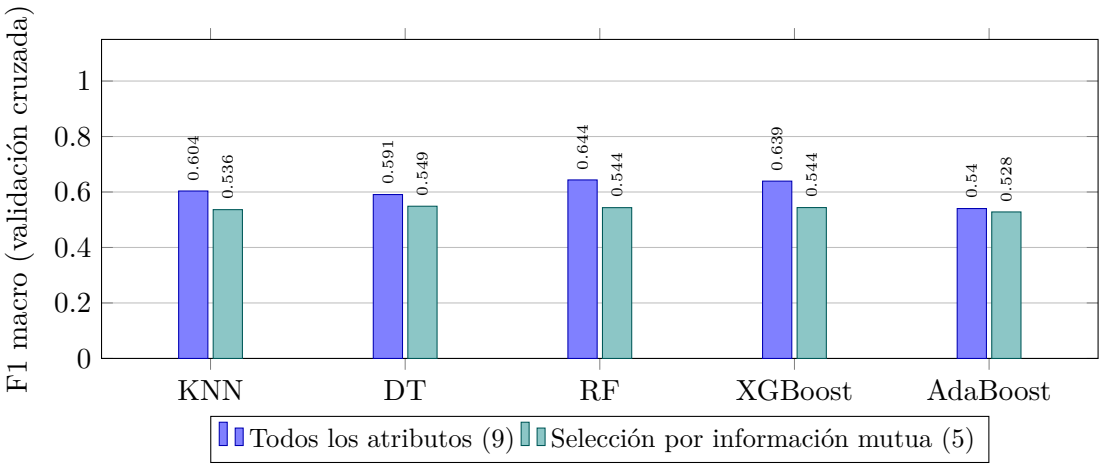


Figura 5: Potabilidad del agua: efecto de la selección de atributos sobre la mejor configuración de cada familia, medido con los mismos cinco pliegues.

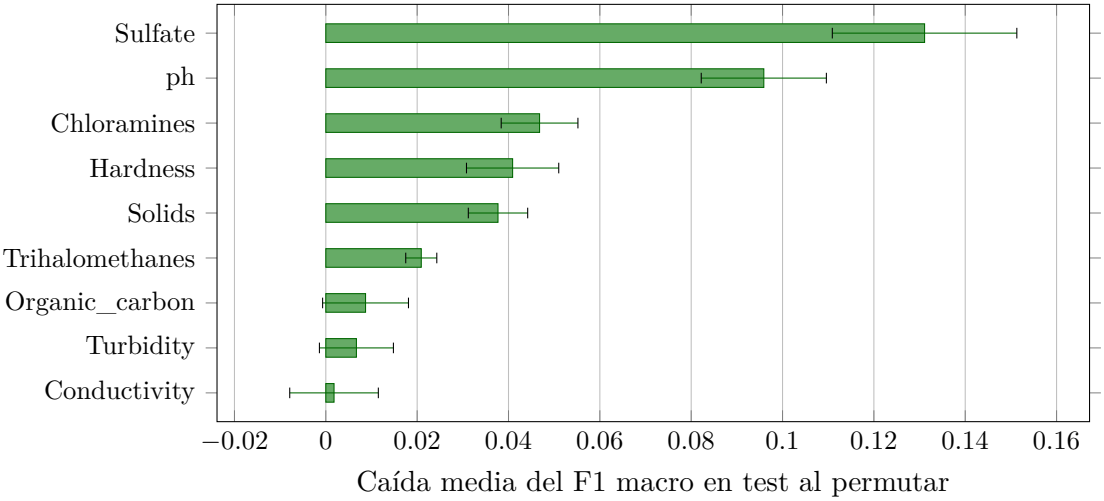


Figura 6: Potabilidad del agua: importancia por permutación del modelo RF (media y desviación de 10 permutaciones de cada variable en test). Es una herramienta de interpretación: no participó en la selección.

Por validación cruzada, el mejor modelo es Random Forest con 500 árboles (F1 macro 0.644 ± 0.022), y también es el mejor en prueba en F1 macro, exactitud y AUC. Aun así, el problema es difícil: las AUC van de 0.615 a 0.727 y ninguna familia detecta más del 50 % de las muestras potables. Hay 37 muestras potables de prueba que los cinco modelos clasifican como no potables. Con medias casi idénticas entre clases, la información útil está en regiones extremas y en combinaciones de variables, y los modelos que mejor la aprovechan son los ensambles de árboles profundos.

La selección de atributos por información mutua empeoró las cinco familias, con caídas de 0.01 (AdaBoost) a 0.10 (RF) en F1 macro (Figura 5). El motivo es instructivo. La información mutua calculada en train asigna 0.026 al sulfato, 0.014 a la dureza y prácticamente 0 al `ph` y a las cloraminas, así que el filtro descarta el `ph`. Sin embargo, la importancia por permutación muestra que el `ph` es la segunda variable más importante para RF. Su efecto aparece sobre todo en combinación con el sulfato (como muestran las reglas del árbol), y un filtro que evalúa cada variable por separado no lo detecta. En este dataset conviene conservar las nueve variables y dejar que los árboles encuentren las interacciones.

5. Resultados: Diabetes

5.1. Datos y partición

Se analizan 390 pacientes con 17 variables: colesterol (`chol`), glucosa estabilizada (`stab.glu`), HDL, razón colesterol/HDL (`ratio`), edad, sexo, localidad, talla, peso, complexión (`frame`), dos mediciones de presión arterial, cintura, cadera y tiempo desde la última comida (`time.ppn`). Sólo el 15.4 % son diabéticos (60 casos): en prueba hay 15 diabéticos, por lo que cada diabético mal clasificado cambia el recall en 6.7 puntos, y en cada pliegue de validación hay unos 9, lo que explica desviaciones de 0.06 a 0.12 en el F1 macro. La glucosa estabilizada separa mucho mejor las clases que cualquier variable de Potabilidad: su mediana es 186 mg/dL en diabéticos y 86.5 mg/dL en no diabéticos. Tras la codificación one-hot hay 21 atributos; la información mutua de la glucosa (0.228) es tres veces la del siguiente, `ratio` (0.063).

Cuadro 21: Procedencia de Diabetes.

Elemento	Valor
Archivo analizado	`diabetes.csv`
Original	`diabetes_original.csv`, 403 filas
Filas analizadas	390
Objetivo	`diabetes`
Predictoras	17
Fuente	`https://hbiostat.org/data/repo/diabetes`
SHA-256 analizado	`47b48720dd246d95528d251c31c27a76` `f44c958a24faf1273b873658f4979e85`

Cuadro 22: Distribución de clases en Diabetes (partición estratificada 75/25, semilla 42).

Clase	Train	Test	Total
No diabetes	247	83	330
Diabetes	45	15	60
Total	292	98	390

Una referencia trivial que siempre predice la clase mayoritaria de train (No diabetes) obtendría en test una exactitud de 84.7 % y un F1 macro de 0.459. Cualquier modelo útil debe superar con claridad ese F1 macro, no sólo la exactitud.

5.2. K vecinos más cercanos (KNN)

Cuadro 23: K vecinos más cercanos (KNN) en Diabetes: validación cruzada estratificada de 5 pliegues sobre train. $\star$ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.7190 ± 0.100	–	89.0 %	0.6808
`k_15`	$k = 15$ vecinos (estándar: 5)	0.6814 ± 0.099	-0.038	88.7 %	0.6424
`pesos_distancia`	votos ponderados por $1/d$ (estándar: uniformes)	0.7190 ± 0.100	+0.000	89.0 %	0.6808
`manhattan`	distancia Manhattan, $p = 1$ (estándar: euclidiana)	0.6358 ± 0.080	-0.083	87.7 %	0.6091
`estandar+seleccion_k11` $\star$	mejor anterior + 11 atributos por información mutua	0.7974 ± 0.070	+0.078	91.8 %	0.7424

Modelo final. Configuración `estandar` con selección de atributos, reajustada con las 292 filas de train. F1 macro: 0.8547 en entrenamiento, 0.7974 en validación cruzada y 0.7478 en test (exactitud 88.8 %, AUC 0.754).

Cuadro 24: K vecinos más cercanos (KNN): matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	Diabetes	No diabetes
Diabetes	7	8
No diabetes	3	80

Cuadro 25: K vecinos más cercanos (KNN): métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
Diabetes	15	0.7000	0.4667	0.5600
No diabetes	83	0.9091	0.9639	0.9357

Análisis. Con todos los atributos, KNN es la familia más débil (F1 macro 0.719 en el estándar). Aumentar a $k = 15$ empeora (0.681): con sólo unos 36 diabéticos en cada ajuste, en un vecindario de 15 suele haber mayoría de no diabéticos y el voto favorece la clase mayoritaria. La distancia Manhattan también empeora (0.636) y ponderar por distancia dio exactamente las mismas métricas que el estándar en los cinco pliegues. El cambio que sí importa es la selección de atributos: con los 11 de mayor información mutua el F1 macro sube a 0.797 (+0.078), la mayor mejora de este dataset. Al eliminar variables poco informativas como la talla, la localidad, el sexo o las segundas mediciones de presión (ausentes en el 65 % de los pacientes e imputadas con la mediana), la distancia pasa a reflejar sobre todo la glucosa. En prueba detecta 7 de 15 diabéticos (recall 0.467) con precisión 0.70. Su AUC (0.754) es la más baja porque, con 5 vecinos, la probabilidad sólo puede tomar seis valores (0, 0.2, . . . , 1), lo que produce una curva ROC escalonada.

5.3. Árbol de decisión (DT)

Cuadro 26: Árbol de decisión (DT) en Diabetes: validación cruzada estratificada de 5 pliegues sobre train. $\star$ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.7826 ± 0.065	–	88.0 %	0.8110
`profundidad_5`	profundidad máxima 5 (estándar: sin límite)	0.7986 ± 0.117	+0.016	89.4 %	0.8101
`hoja_minima_10`	mínimo 10 observaciones por hoja (estándar: 1)	0.8064 ± 0.070	+0.024	90.1 %	0.8232
`clases_balanceadas`	pesos por observación inversos a la frecuencia de su clase	0.7510 ± 0.077	-0.032	87.7 %	0.7454
`hoja_minima_10+ seleccion_k11` $\star$	mejor anterior + 11 atributos por información mutua	0.8160 ± 0.085	+0.033	90.8 %	0.8182

Modelo final. Configuración `hoja_minima_10` con selección de atributos, reajustada con las 292 filas de train. F1 macro: 0.8894 en entrenamiento, 0.8160 en validación cruzada y 0.7783 en test (exactitud 89.8 %, AUC 0.907).

Cuadro 27: Árbol de decisión (DT): matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	Diabetes	No diabetes
Diabetes	8	7
No diabetes	3	80

Cuadro 28: Árbol de decisión (DT): métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
Diabetes	15	0.7273	0.5333	0.6154
No diabetes	83	0.9195	0.9639	0.9412

```
|--- stab.glu <= 158.00
|   |--- stab.glu <= 107.50
|   |   |--- age <= 63.50
|   |   |   |--- clase: No diabetes [n=183] (rama truncada)
|   |   |--- age > 63.50
|   |   |   |--- clase: No diabetes [n=30] (rama truncada)
|   |--- stab.glu > 107.50
|   |   |--- chol <= 243.50
|   |   |   |--- clase: No diabetes [n=33] (rama truncada)
|   |   |--- chol > 243.50
|   |   |   |--- clase: Diabetes [n=11]
|--- stab.glu > 158.00
|   |--- stab.glu <= 201.50
|   |   |--- clase: Diabetes [n=13]
|   |--- stab.glu > 201.50
|   |   |--- ratio <= 5.45
|   |   |   |--- clase: Diabetes [n=10]
|   |   |--- ratio > 5.45
|   |   |   |--- clase: Diabetes [n=12]
```

Figura 7: Primeros tres niveles del árbol final (profundidad total 5, 11 hojas). Los umbrales están en las unidades originales: la estandarización se invirtió con la media y desviación de train. n es la cantidad de observaciones de train en el nodo.

Análisis. Exigir al menos 10 observaciones por hoja es de nuevo la mejor variante (0.806, +0.024), igual que en Potabilidad. Limitar la profundidad a 5 da un resultado parecido (0.799) pero con una variación entre pliegues mucho mayor (0.117 frente a 0.070). Ponderar las clases empeora (0.751): con tan pocos diabéticos, darles más peso hace que el árbol cree hojas para casos aislados. La selección de atributos mejora un poco (0.816) y es la versión final. El árbol resultante es

pequeño (profundidad 5, 11 hojas) y fácil de leer: su primera regla es `stab.glu` > 158 mg/dL ⇒ diabetes, y en el rango de 107.5 a 158 mg/dL marca como diabéticos a los pacientes con colesterol mayor que 243.5 mg/dL. La importancia por permutación confirma que en prueba el árbol depende exclusivamente de la glucosa (caída de 0.252 en F1 macro; el resto, cero). En prueba detecta 8 de 15 diabéticos (recall 0.533, precisión 0.727) con exactitud 89.8 % y AUC 0.907. El umbral de 158 mg/dL lo aprendió el árbol de estos datos; no es un criterio diagnóstico. Como referencia, el criterio clínico para una glucosa casual es 200 mg/dL con síntomas [24].

5.4. Random Forest (RF)

Cuadro 29: Random Forest (RF) en Diabetes: validación cruzada estratificada de 5 pliegues sobre train. ⋆ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.8364 ± 0.087	–	92.5 %	0.8101
`500_arboles`	500 árboles (estándar: 100)	0.8177 ± 0.079	-0.019	91.8 %	0.7878
`hoja_minima_5`	mínimo 5 observaciones por hoja (estándar: 1)	0.7633 ± 0.068	-0.073	90.1 %	0.7232
`clases_balanceadas`	pesos por observación inversos a la frecuencia de su clase	0.8162 ± 0.081	-0.020	91.1 %	0.8111
`estandar+seleccion_k11` ⋆	mejor anterior + 11 atributos por información mutua	0.8398 ± 0.066	+0.003	92.5 %	0.8192

Modelo final. Configuración `estandar` con selección de atributos, reajustada con las 292 filas de train. F1 macro: 1.0000 en entrenamiento, 0.8398 en validación cruzada y 0.7638 en test (exactitud 88.8 %, AUC 0.892).

Cuadro 30: Random Forest (RF): matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	Diabetes	No diabetes
Diabetes	8	7
No diabetes	4	79

Cuadro 31: Random Forest (RF): métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
Diabetes	15	0.6667	0.5333	0.5926
No diabetes	83	0.9186	0.9518	0.9349

Análisis. En validación cruzada, Random Forest es el mejor algoritmo del dataset. Su configuración estándar (0.836) supera a las tres variantes: 500 árboles no mejora (0.818), la ponderación de clases tampoco (0.816) y exigir 5 observaciones por hoja empeora claramente (0.763), porque impide aislar a los pocos diabéticos en hojas propias. Con selección de atributos alcanza 0.840 y una desviación menor (0.066 frente a 0.087), así que esa es la versión final. En prueba tiene exactitud 88.8 %, F1 macro 0.764 y AUC 0.892; detecta 8 de 15 diabéticos y tiene un falso positivo más que DT, XGBoost y AdaBoost (4 en lugar de 3). Por eso su F1 de prueba es ligeramente menor, aunque fue el elegido por validación. Esa diferencia de un paciente está muy por debajo de la incertidumbre del experimento. La importancia interna muestra que, además de la glucosa (0.454), el bosque reparte peso entre `ratio`, HDL y edad, pero en prueba sólo la glucosa produce una caída grande al permutarse.

5.5. XGBoost

Cuadro 32: XGBoost en Diabetes: validación cruzada estratificada de 5 pliegues sobre train. ⋆ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.7921 ± 0.089	–	90.1 %	0.7869
`tasa_0_05` ⋆	tasa de aprendizaje 0.05 (estándar: 0.3)	0.8208 ± 0.117	+0.029	91.8 %	0.8060
`profundidad_3`	profundidad máxima 3 (estándar: 6)	0.8041 ± 0.098	+0.012	90.8 %	0.8000
`clases_balanceadas`	pesos por observación inversos a la frecuencia de su clase	0.7885 ± 0.102	-0.004	89.4 %	0.8010
`tasa_0_05+seleccion_k11`	mejor anterior + 11 atributos por información mutua	0.8034 ± 0.108	+0.011	91.1 %	0.7929

Modelo final. Configuración `tasa_0_05` con todos los atributos, reajustada con las 292 filas de train. F1 macro: 0.9934 en entrenamiento, 0.8208 en validación cruzada y 0.7783 en test (exactitud 89.8 %, AUC 0.929).

Cuadro 33: XGBoost: matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	Diabetes	No diabetes
Diabetes	8	7
No diabetes	3	80

Cuadro 34: XGBoost: métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
Diabetes	15	0.7273	0.5333	0.6154
No diabetes	83	0.9195	0.9639	0.9412

Análisis. Bajar la tasa de aprendizaje a 0.05 es la mejor variante (0.821, +0.029): con sólo 292 pacientes de entrenamiento, que cada árbol aporte menos funciona como regularización. Reducir la profundidad a 3 ayuda algo menos (0.804) y ponderar las clases no ayuda (0.789). A diferencia de las otras familias, la selección de atributos empeora a XGBoost (0.803): el algoritmo ya elige en cada división la variable más útil y quitarle variables secundarias le resta poco ruido y algo de información. En prueba obtiene la mejor AUC del estudio (0.929), exactitud 89.8 % y F1 macro 0.778, con la misma matriz de confusión que DT y AdaBoost. Su F1 de entrenamiento (0.993) indica que aun con tasa baja casi memoriza el entrenamiento.

5.6. AdaBoost

Cuadro 35: AdaBoost en Diabetes: validación cruzada estratificada de 5 pliegues sobre train. ⋆ = configuración elegida; Δ = diferencia de F1 macro frente al estándar.

Configuración	Cambio respecto del estándar	F1 macro CV	Δ	Exact. CV	Recall CV
`estandar`	valores predeterminados de la biblioteca	0.7920 ± 0.058	–	90.4 %	0.7616
`200_estimadores`	200 estimadores (estándar: 50)	0.7896 ± 0.079	-0.002	90.4 %	0.7616
`tasa_0_5`	tasa de aprendizaje 0.5 (estándar: 1.0)	0.7994 ± 0.122	+0.007	91.1 %	0.7838
`clases_balanceadas`	pesos por observación inversos a la frecuencia de su clase	0.7944 ± 0.063	+0.002	89.4 %	0.8009
`tasa_0_5+seleccion_k11` ⋆	mejor anterior + 11 atributos por información mutua	0.8027 ± 0.087	+0.011	91.1 %	0.7747

Modelo final. Configuración `tasa_0_5` con selección de atributos, reajustada con las 292 filas de train. F1 macro: 0.8865 en entrenamiento, 0.8027 en validación cruzada y 0.7783 en test (exactitud 89.8 %, AUC 0.922).

Cuadro 36: AdaBoost: matriz de confusión en test (filas: real; columnas: predicción).

Real / pred.	Diabetes	No diabetes
Diabetes	8	7
No diabetes	3	80

Cuadro 37: AdaBoost: métricas por clase en test.

Clase	Soporte	Precisión	Recall	F1
Diabetes	15	0.7273	0.5333	0.6154
No diabetes	83	0.9195	0.9639	0.9412

Análisis. Las variantes de AdaBoost quedan todas entre 0.790 y 0.799, sin diferencias claras. La tasa 0.5 es la de mayor media (0.799), pero también la de mayor desviación (0.122). La ponderación de clases logra el mayor recall macro de validación (0.801), aunque no el mayor F1. Con selección de atributos sube a 0.803 y esa es la versión final. En prueba coincide con DT y XGBoost (exactitud 89.8 %, F1 macro 0.778, 8 de 15 diabéticos) y tiene AUC 0.922. Es el ensamble con menor sobreajuste (F1 0.887 en entrenamiento frente a 0.778 en prueba). En este dataset los stumps sí funcionan, a diferencia de Potabilidad: un único corte en la glucosa ya separa bien las clases, y los siguientes stumps afinan con el colesterol (importancia interna 0.120) y el peso (0.106).

5.7. Comparación de los cinco modelos elegidos

Cuadro 38: Diabetes: resumen de los ganadores. La selección usó sólo la columna F1 CV; el mejor algoritmo por validación es RF.

Familia	Configuración	F1 train	F1 CV	F1 test	Exactitud test	Recall Diabetes	AUC
KNN	`estandar` + sel.	0.8547	0.7974	0.7478	88.8 %	0.4667	0.754
DT	`hoja_minima_10` + sel.	0.8894	0.8160	0.7783	89.8 %	0.5333	0.907
RF	`estandar` + sel.	1.0000	0.8398	0.7638	88.8 %	0.5333	0.892
XGBoost	`tasa_0_05`	0.9934	0.8208	0.7783	89.8 %	0.5333	0.929
AdaBoost	`tasa_0_5` + sel.	0.8865	0.8027	0.7783	89.8 %	0.5333	0.922

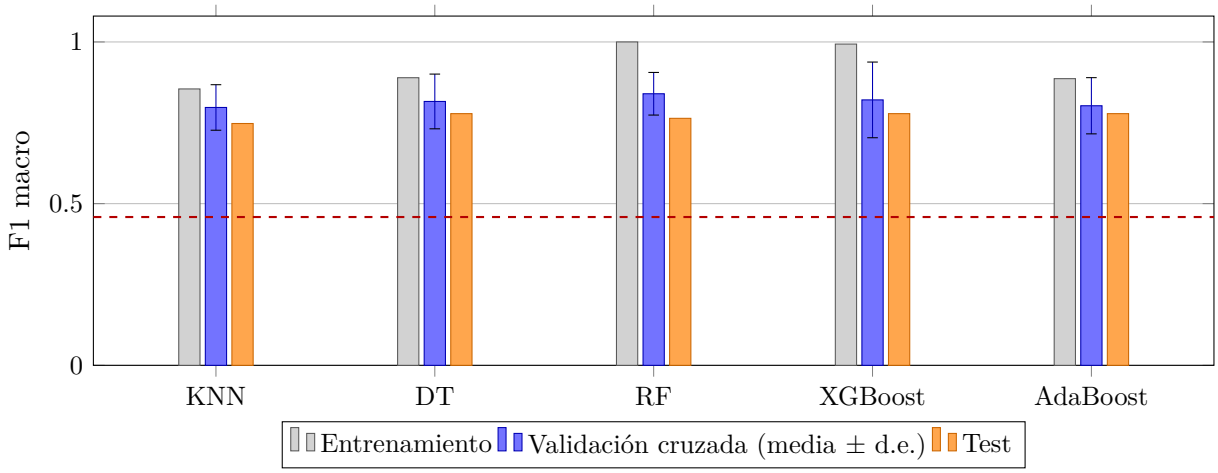


Figura 8: Diabetes: F1 macro del modelo elegido en cada familia. La línea roja es la referencia que siempre predice la clase mayoritaria (F1 macro 0.459). La distancia entre la barra gris y la azul indica sobreajuste.

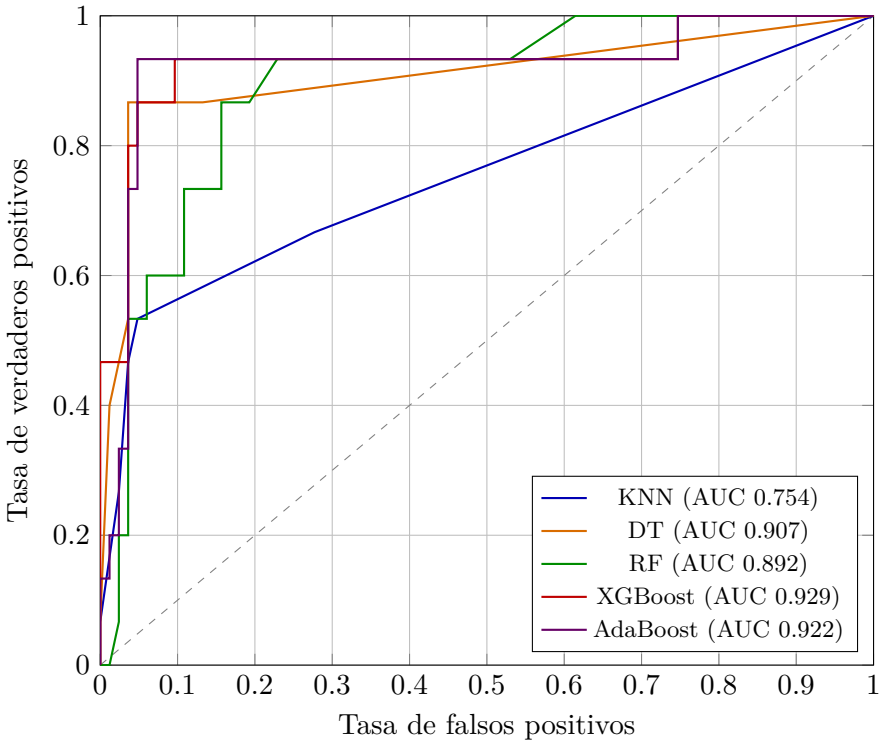


Figura 9: Diabetes: curvas ROC en test de los cinco modelos elegidos; clase positiva: Diabetes. La diagonal corresponde a un clasificador aleatorio.

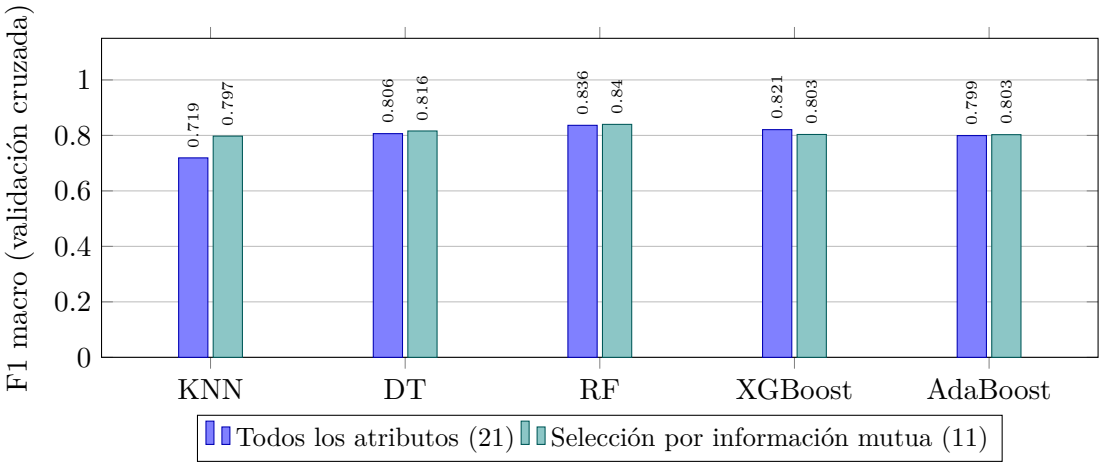


Figura 10: Diabetes: efecto de la selección de atributos sobre la mejor configuración de cada familia, medido con los mismos cinco pliegues.

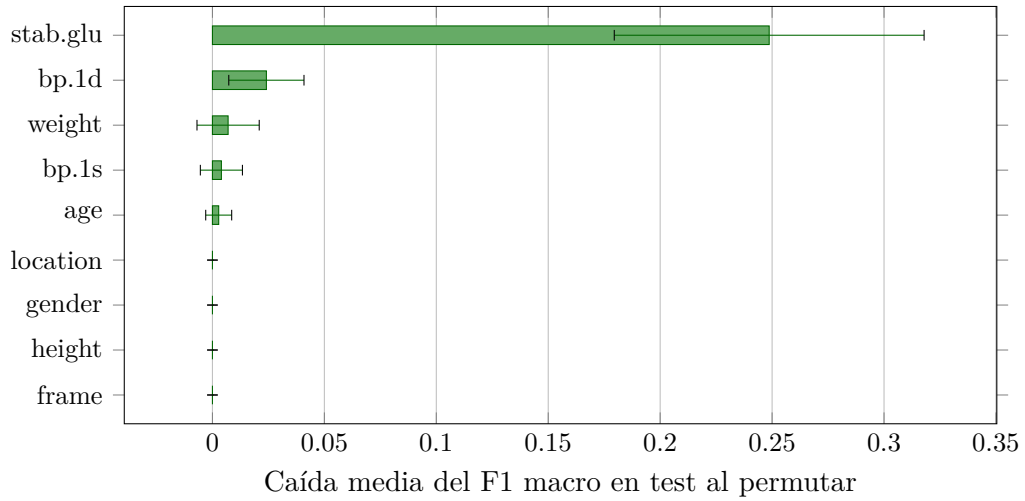


Figura 11: Diabetes: importancia por permutación del modelo RF (media y desviación de 10 permutaciones de cada variable en test). Es una herramienta de interpretación: no participó en la selección.

Los cuatro métodos basados en árboles quedan entre 0.80 y 0.84 de F1 macro en validación cruzada, con desviaciones de 0.07 a 0.12. Las diferencias entre ellos son menores que esa variación, así que el orden no es concluyente; RF se eligió por tener la mayor media. En prueba todos superan claramente la referencia (F1 macro 0.459) y DT, XGBoost y AdaBoost producen la misma matriz de confusión: 8 diabéticos detectados, 7 no detectados y 3 falsos positivos.

Los 7 diabéticos que no detectan DT, RF, XGBoost y AdaBoost son exactamente los mismos (KNN falla esos 7 y uno más), y todos tienen glucosa estabilizada entre 90 y 130 mg/dL, mientras que los 8 detectados tienen 196 mg/dL o más. La etiqueta se definió con la hemoglobina glicosilada, que refleja la glucosa promedio de los últimos dos o tres meses, y puede estar elevada aunque una medición puntual de glucosa sea normal. Como `glyhb` se excluyó para evitar fuga, esos pacientes no se distinguen de los no diabéticos con las variables disponibles. Es un límite de la información, no de los algoritmos: ningún ajuste de hiperparámetros los recuperaría sin aumentar mucho los falsos positivos.

Las AUC de 0.89 a 0.93 indican que los modelos ordenan bien a los pacientes por riesgo. Bajar el umbral de decisión de 0.5 aumentaría el recall de diabetes a cambio de más falsos positivos. Ese umbral debería elegirse con validación cruzada y no con el conjunto de prueba, por lo que se deja como trabajo futuro. A diferencia de Potabilidad, aquí la selección de atributos ayudó a cuatro de cinco familias (Figura 10): la información relevante se concentra en una variable con relación monótona con la clase, y la información mutua la detecta sin problema.

6. Discusión comparativa

6.1. Qué variantes ganaron y por qué

La Tabla 39 resume la configuración elegida en cada caso. Hay dos patrones que se repiten en ambos datasets. Para el árbol de decisión, exigir un mínimo de 10 observaciones por hoja fue siempre la mejor variante: un árbol sin límites memoriza el entrenamiento, y limitar el tamaño de las hojas reduce esa varianza sin eliminar las reglas útiles. Para KNN, aumentar k a 15 no fue la mejor variante en ningún dataset: en Diabetes bajó el F1 macro de 0.719 a 0.681, porque con 45 diabéticos en train un vecindario de 15 casos queda casi siempre dominado por la clase mayoritaria; en Potabilidad subió apenas 0.011, menos que el cambio a distancia Manhattan.

En cambio, otras decisiones dependen del dataset. Ponderar las clases fue la mejor variante de XGBoost y AdaBoost en Potabilidad, donde los modelos tienden a predecir "no potable". En Diabetes no fue la mejor variante en ninguna familia y bajó el F1 de DT (0.783 a 0.751) y RF (0.836 a 0.816): dar más peso a 45 diabéticos lleva a los árboles a crear reglas para casos aislados. La selección de atributos mejoró cuatro de cinco familias en Diabetes (en RF sólo 0.003, una diferencia despreciable) y empeoró las cinco en Potabilidad. La explicación es la forma de la relación entre variables y clase: en Diabetes la señal está casi toda en la glucosa y es monótona (más glucosa, más riesgo), así que un filtro univariado la encuentra; en Potabilidad la señal está en los extremos y en la combinación de sulfato y `ph`, y el filtro descarta el `ph` aunque sea la segunda variable más importante para el mejor modelo.

Cuadro 39: Configuración elegida por validación cruzada en cada familia y dataset.

Familia	Potabilidad	Diabetes
KNN	Manhattan ($p = 1$)	estándar + selección de 11 atributos
DT	mínimo 10 por hoja	mínimo 10 por hoja + selección
RF	500 árboles	estándar + selección
XGBoost	clases balanceadas	tasa de aprendizaje 0.05
AdaBoost	clases balanceadas	tasa 0.5 + selección

6.2. Sobreajuste y estabilidad

Las Figuras 3 y 8 comparan el F1 de entrenamiento, validación cruzada y prueba. Random Forest y XGBoost llegan a F1 de entrenamiento de 1.0 o casi 1.0 en ambos datasets: memorizan el entrenamiento. Aun así, RF es el que mejor generaliza en Potabilidad, porque el promedio de cientos de árboles cancela buena parte de sus errores individuales. AdaBoost con stumps es el extremo opuesto en Potabilidad: casi no sobreajusta (0.575 en entrenamiento y 0.582 en prueba), pero subajusta. KNN y DT quedan en medio.

La validación cruzada también muestra cuánto se puede confiar en cada diferencia. En Potabilidad (unos 300 casos por pliegue) las desviaciones de los ganadores están entre 0.02 y 0.05, y la ventaja de RF sobre KNN, DT y AdaBoost es clara; frente a XGBoost (0.639) la diferencia es menor que la desviación. En Diabetes (unos 58 casos y 9 diabéticos por pliegue) las desviaciones llegan a 0.12 y las cuatro familias de árboles son estadísticamente indistinguibles. Una sola partición de validación, como la que se usaba antes en el framework, habría dado un ganador sin advertir esa incertidumbre.

6.3. Exactitud frente a F1 macro

En Diabetes la exactitud de prueba (88.8–89.8 %) parece alta, pero la referencia que siempre predice "no diabetes" ya obtiene 84.7 %. El F1 macro (0.75–0.78 frente a 0.459) y el recall de diabetes (0.47–0.53) muestran mejor lo que hace el modelo: detecta la mitad de los diabéticos. En

Potabilidad, AdaBoost estándar tiene una exactitud de validación de 59.7 %, igual a la proporción de la clase mayoritaria: sin el F1 macro se habría pasado por alto que el modelo casi no aprendió nada.

6.4. Limitaciones

- Se usó una única partición de prueba (503 y 98 casos). Con 15 diabéticos en prueba, un solo paciente cambia el recall en 6.7 puntos. No se calcularon intervalos de confianza ni pruebas de significancia.
- El catálogo tiene tres variantes por familia y cada una cambia un solo hiperparámetro. Es deliberado, para poder atribuir los efectos, pero no explora combinaciones (por ejemplo, más árboles y hojas mínimas a la vez).
- Todos los modelos usan el umbral de probabilidad 0.5. Ajustarlo con validación cruzada podría aumentar el recall de la clase minoritaria.
- La clase de Diabetes se construyó con el criterio histórico glyhb > 7, y la versión de Potabilidad usada son los casos completos del archivo público. Los resultados son educativos: no constituyen un diagnóstico médico ni una certificación de potabilidad.
- La importancia por permutación mide de qué depende el modelo, no relaciones causales.

7. Conclusiones

1. Los cinco algoritmos quedaron integrados en la clase `Clasificacion` del framework, con un protocolo reproducible: preparación aprendida sólo con entrenamiento, validación cruzada estratificada de 5 pliegues para elegir y una única evaluación en prueba. Cada experimento se repite con un comando y deja toda la evidencia.
2. En Potabilidad, el mejor modelo es Random Forest con 500 árboles (F1 macro de validación 0.644 ± 0.022; en prueba, F1 macro 0.659, exactitud 70.2 % y AUC 0.727). Supera con claridad la referencia (59.6 % de exactitud) y a las demás familias, pero detecta sólo el 43 % de las muestras potables. Las variables apenas difieren en promedio entre clases y la señal aparece en los extremos y en la combinación de sulfato y `ph`, algo que los ensambles de árboles profundos captan mejor que KNN o AdaBoost con stumps.
3. En Diabetes, DT, RF, XGBoost y AdaBoost obtienen resultados equivalentes dentro de la variación de la validación cruzada (F1 macro 0.80–0.84). RF fue el elegido por validación; en prueba los cuatro detectan 8 de 15 diabéticos con AUC entre 0.89 y 0.93. La glucosa estabilizada explica casi todo el poder predictivo, y los siete diabéticos que ningún modelo detecta tienen glucosa normal en la medición, lo que marca un límite de la información disponible.
4. Las variantes que mejoraron dependen del algoritmo y del dataset. Exigir hojas de al menos 10 observaciones mejoró el árbol de decisión en ambos datasets, pero limitar las hojas de Random Forest empeoró el bosque, porque el promedio ya controla la varianza. Ponderar las clases ayudó a los métodos de boosting cuando tendían a ignorar la clase minoritaria (Potabilidad). Seleccionar atributos por información mutua ayudó cuando la señal estaba en pocas variables con relación monótona (Diabetes) y perjudicó cuando dependía de interacciones (Potabilidad).
5. La exactitud sola es engañosa con clases desbalanceadas. El F1 macro, el recall por clase y la comparación con una referencia trivial fueron necesarios para interpretar los resultados.

Como trabajo futuro se propone elegir el umbral de decisión con validación cruzada para aumentar el recall de la clase minoritaria, explorar combinaciones de hiperparámetros con búsqueda en rejilla dentro de la validación cruzada y repetir el experimento con varias semillas para estimar la estabilidad de las conclusiones.

Referencias

[1] T. Hastie, R. Tibshirani y J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2.ª ed. Springer, 2009.

[2] T. M. Cover y P. E. Hart. Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967. https://doi.org/10.1109/TIT.1967.1053964.

[3] scikit-learn. *Nearest Neighbors*, documentación oficial. https://scikit-learn.org/stable/modules/neighbors.html.

[4] scikit-learn. *Decision Trees*, documentación oficial. https://scikit-learn.org/stable/modules/tree.html.

[5] L. Breiman. Random Forests. *Machine Learning*, 45:5–32, 2001. https://doi.org/10.1023/A:1010933404324.

[6] scikit-learn. *Ensembles: Gradient boosting, random forests, bagging, voting, stacking*, documentación oficial. https://scikit-learn.org/stable/modules/ensemble.html.

[7] T. Chen y C. Guestrin. XGBoost: A Scalable Tree Boosting System. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. https://doi.org/10.1145/2939672.2939785.

[8] XGBoost. *Introduction to Boosted Trees*, documentación oficial. https://xgboost.readthedocs.io/en/stable/tutorials/model.html.

[9] Y. Freund y R. E. Schapire. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997. https://doi.org/10.1006/jcss.1997.1504.

[10] scikit-learn. *AdaBoostClassifier*, documentación oficial. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html.

[11] scikit-learn. *Metrics and scoring: quantifying the quality of predictions*, documentación oficial. https://scikit-learn.org/stable/modules/model_evaluation.html.

[12] scikit-learn. *Common pitfalls and recommended practices: Data leakage*, documentación oficial. https://scikit-learn.org/stable/common_pitfalls.html.

[13] R. Kohavi. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. En *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 1137–1145, 1995.

[14] scikit-learn. *Cross-validation: evaluating estimator performance*, documentación oficial. https://scikit-learn.org/stable/modules/cross_validation.html.

[15] A. Kraskov, H. Stögbauer y P. Grassberger. Estimating mutual information. *Physical Review E*, 69:066138, 2004. https://doi.org/10.1103/PhysRevE.69.066138.

[16] B. C. Ross. Mutual Information between Discrete and Continuous Data Sets. *PLoS ONE*, 9(2):e87357, 2014. https://doi.org/10.1371/journal.pone.0087357.

[17] I. Guyon y A. Elisseeff. An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003.

[18] H. He y E. A. Garcia. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. https://doi.org/10.1109/TKDE.2008.239.

[19] T. Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. https://doi.org/10.1016/j.patrec.2005.10.010.

[20] scikit-learn. *Permutation feature importance*, documentación oficial. https://scikit-learn.org/stable/modules/permutation_importance.html.

[21] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[22] A. Kadiwal. *Water Quality* (water_potability.csv). Kaggle, 2021. https://www.kaggle.com/datasets/adityakadiwal/water-potability.

[23] Vanderbilt University, Department of Biostatistics. *Diabetes Dataset* (J. P. Willems, J. T. Saunders, D. E. Hunt y J. B. Schorling). https://hbiostat.org/data/repo/diabetes.

[24] American Diabetes Association Professional Practice Committee. 2. Diagnosis and Classification of Diabetes: Standards of Care in Diabetes—2024. *Diabetes Care*, 47(Suppl. 1):S20–S42, 2024. https://doi.org/10.2337/dc24-S002.

A. Reproducción de los experimentos

Desde la carpeta raíz del proyecto, con Python 3.11 o superior y las dependencias de `requirements-lab02-lock.txt` instaladas, los pasos son los siguientes. Cada experimento tarda alrededor de un minuto. La carpeta de salida debe estar vacía: el script no sobrescribe evidencia anterior.

```
python -m pip install -r requirements-lab02-lock.txt

# 1. Preparación de los datos (determinista; conserva los originales)
python -m scripts.preparar_potabilidad_lab02 data/lab02/water_potability.csv
python -m scripts.preparar_diabetes_lab02 data/lab02/diabetes_original.csv

# 2. Experimentos (validación cruzada de 5 pliegues, semilla 42)
python -m scripts.ejecutar_lab02 data/lab02/potabilidad.csv --target Potability
    --salida salida_lab02/potabilidad --semilla 42
    --fuente https://www.kaggle.com/datasets/adityakadiwal/water-potability
python -m scripts.ejecutar_lab02 data/lab02/diabetes.csv --target diabetes
    --features chol stab.glu hdl ratio location age gender height weight
    frame bp.1s bp.1d bp.2s bp.2d waist hip time.ppn --incluir-categoricas
    --salida salida_lab02/diabetes --semilla 42
    --fuente https://hbiostat.org/data/repo/diabetes

# 3. Informe y pruebas del software
python -m scripts.generar_informe_lab02 salida_lab02 --autor "Nombre (ID)"
python -m pytest -q
```

Las líneas indentadas continúan el comando anterior (en PowerShell se escriben en una sola línea). Cada carpeta de resultados contiene `configuracion.json` (argumentos, SHA-256 del CSV, versiones, índices de train, test y de cada pliegue), `comparacion_validacion.csv` (todas las configuraciones con media, desviación y puntuación por pliegue), `mejores_modelos.json`, `ranking_atributos.csv` y, por algoritmo, la matriz de confusión, las predicciones de prueba, la curva ROC y la importancia por permutación; además, `DT/arbol.txt` con las reglas del árbol.

Cuadro 40: Versiones registradas durante la ejecución.

Paquete	Versión
numpy	2.5.3
pandas	3.0.5
scikit-learn	1.9.1
xgboost	3.4.1
python	3.13.16

Cuadro 41: Huellas de los datos utilizados.

Archivo	SHA-256
water_potability.csv	`904004bde729bfe3d2e195f46343bceead09e32a0eb95bb8184e7e20e029b2bf`
potabilidad.csv	`d2569bece50025b9dbf21cfd720ecbedcdaa838ca082db29ef853c5fb9da8e12`
diabetes_original.csv	`3de140ba6771ee5af7043500065c667cfb5c059eb840cc22e14cabf1ebfc0838`
diabetes.csv	`47b48720dd246d95528d251c31c27a76f44c958a24faf1273b873658f4979e85`